

視窗程式設計 (II)

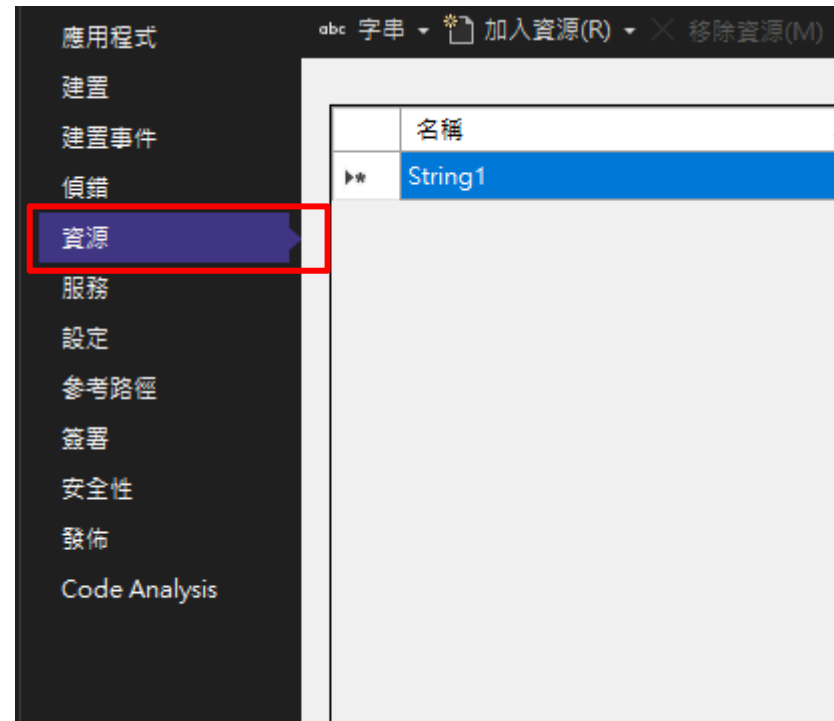
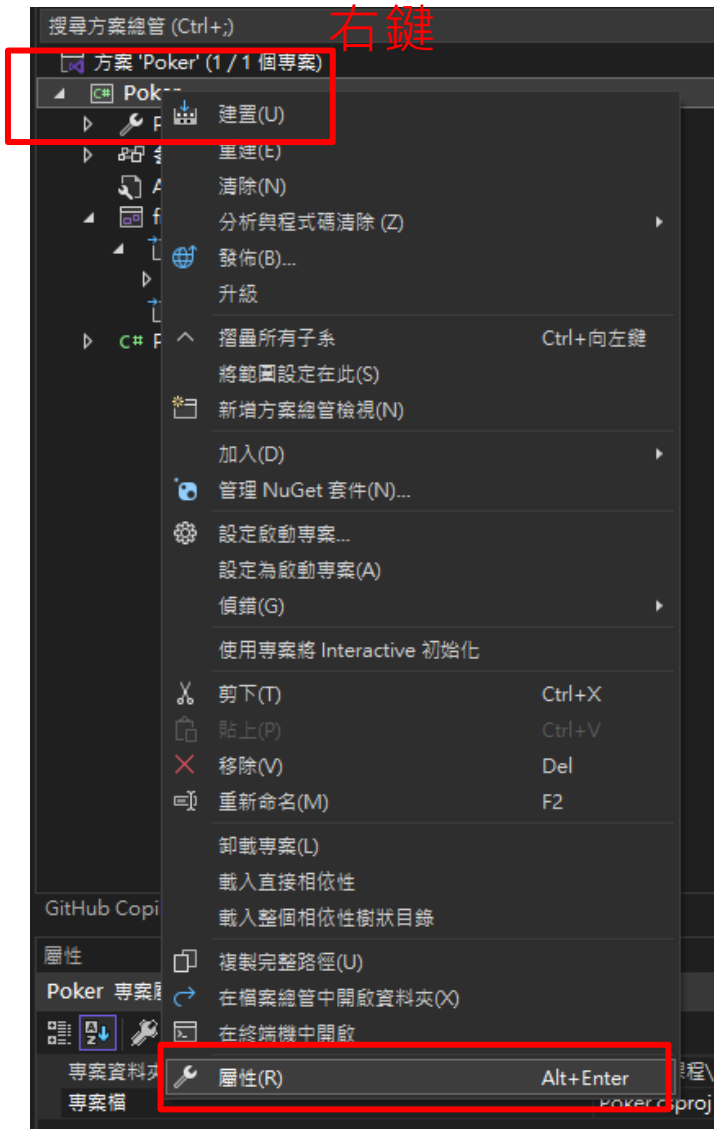
Windows Programming (II)

專案圖片資源和動態控制項

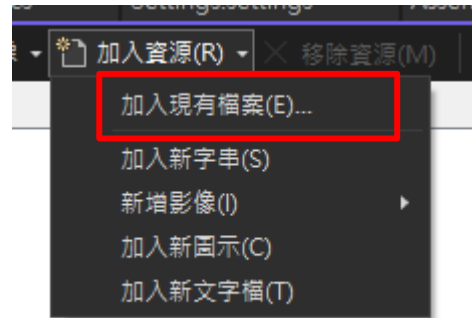
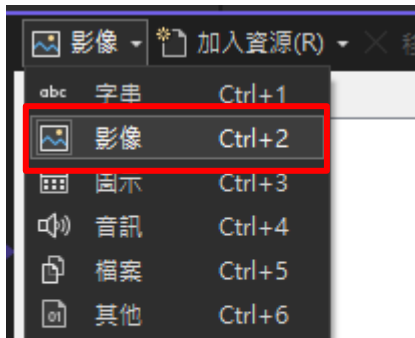
Section 4.1

專案圖片資源

加入專案圖片資源



加入專案圖片資源



選擇撲克牌圖案



建置

建置事件

偵錯

資源*

服務

設定

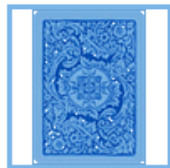
參考路徑

簽署

安全性

發佈

Code Analysis



back



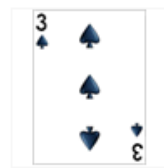
pic1



pic10



pic11



pic12



pic13



pic14



pic15



pic16



pic17



pic18



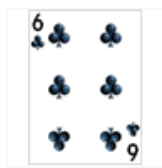
pic19



pic2



pic20



pic21



pic22



pic23



pic24



pic25



pic26



pic27



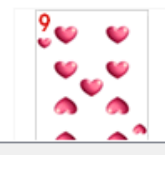
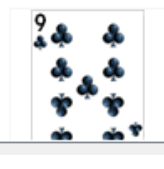
pic28



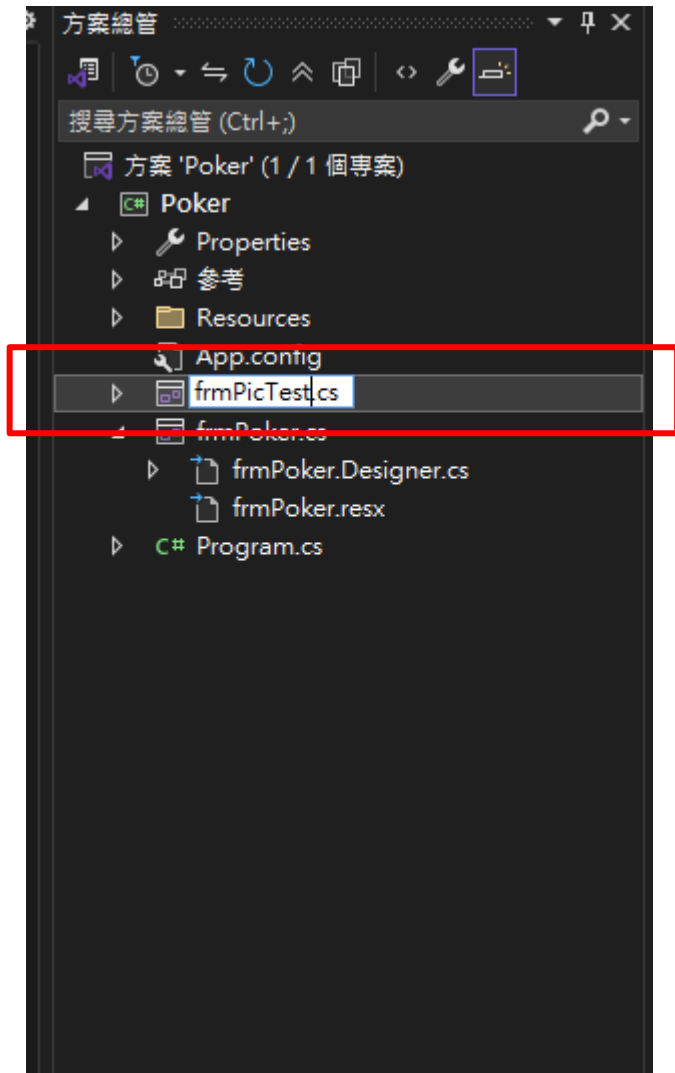
pic29



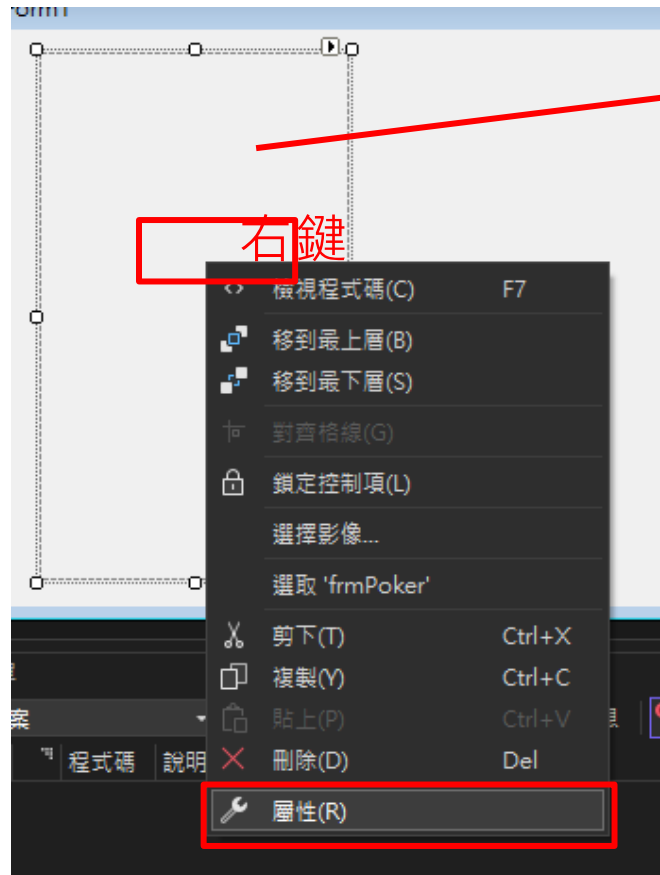
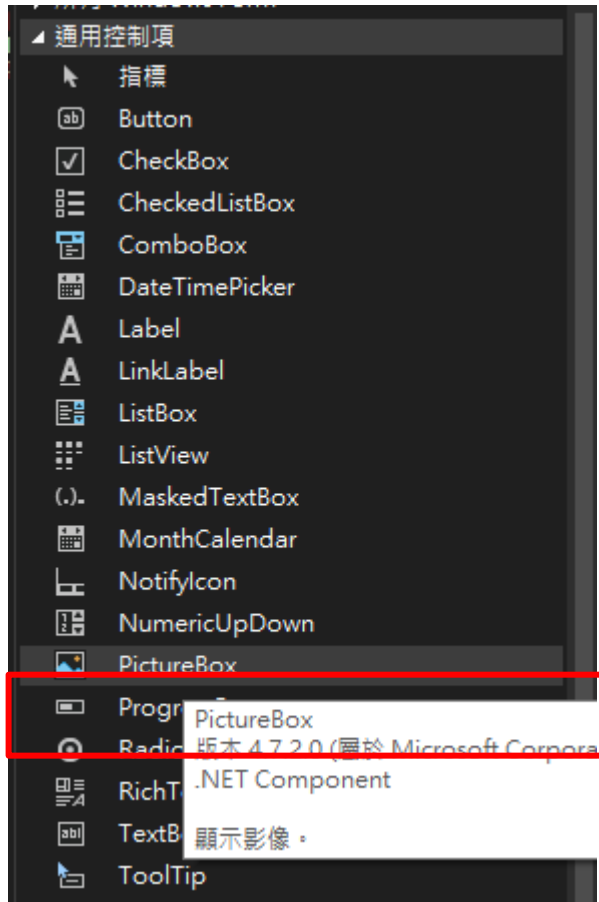
pic3



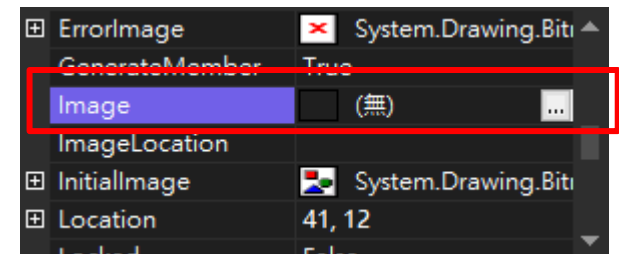
新增 frmPicTest 表單



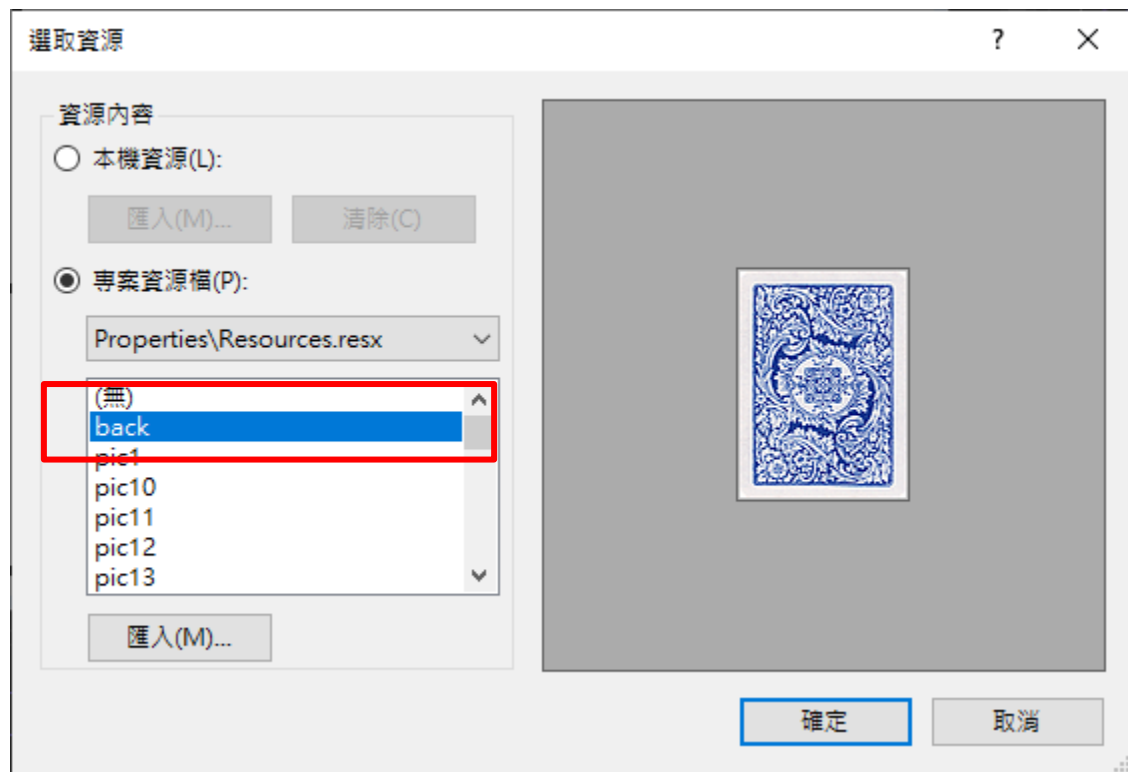
新增 PictureBox



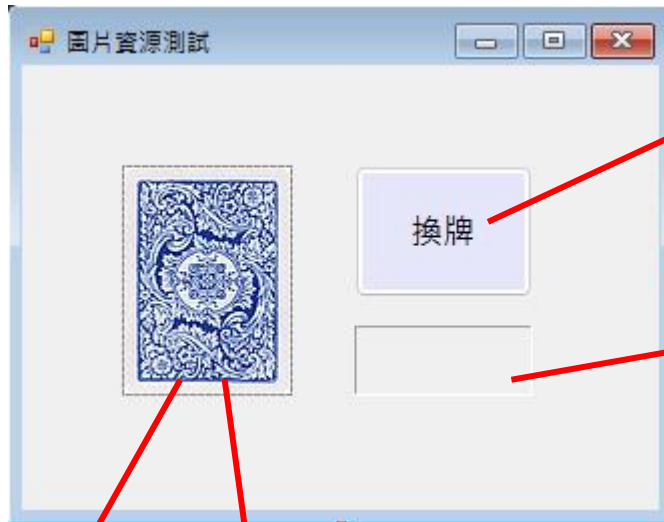
picTest



選擇牌背圖案



控制項設定



picTest

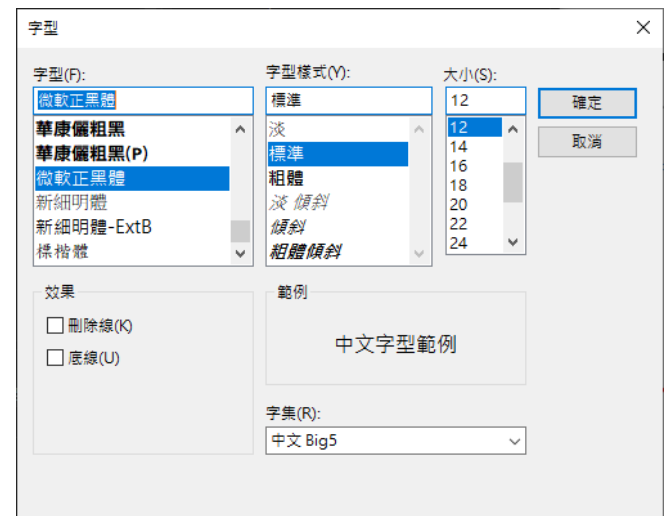
Modifiers	Private
Padding	0, 0, 0, 0
Size	85, 115
SizeMode	AutoSize
Tag	
UseWaitCursor	False
Visible	True

btnTest

AutoSize	False
AutoSizeMode	GrowOnly
BackColor	Lavender
Backgroundimage	(無)

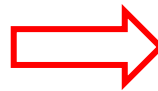
lblNum

BackColor	Control
BorderStyle	Fixed3D
CausesValidation	True
ContextMenuStrip	(無)



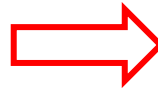
按下換牌顯示紅心10

```
private void btnTest_Click(object sender, EventArgs e)
{
    this.picTest.Image = PicTest.Properties.Resources.pic39;
}
```



利用字串取得圖片資源

```
private void btnTest_Click(object sender, EventArgs e) {  
    picTest.Image =  
    PicTest.Properties.Resources.ResourceManager.GetObject("pic21  
") as Image;  
}
```



隨機亂數取得圖片資源

```
private void btnTest_Click(object sender, EventArgs e) {  
    Random random = new Random();  
  
    int r = random.Next(1, 53);  
  
    picTest.Image = PicTest.Properties.Resources.  
        ResourceManager.GetObject($"pic{r}")  
        as Image;  
  
    lblNum.Text = r.ToString();  
}
```



GetImage 函式

// 用字串讀取圖片

```
private Image GetImage(string name) {  
    return PicTest.Properties.Resources.ResourceManager  
        .GetObject(name) as Image;  
}
```

// 用撲克牌編號讀取圖片

```
private Image GetImage(int num) {  
    return GetImage($"pic{r}");  
}
```

當換牌鍵按下

```
private void btnTest_Click(object sender, EventArgs e)
{
    Random random = new Random();

    int picNum = random.Next(1, 53);

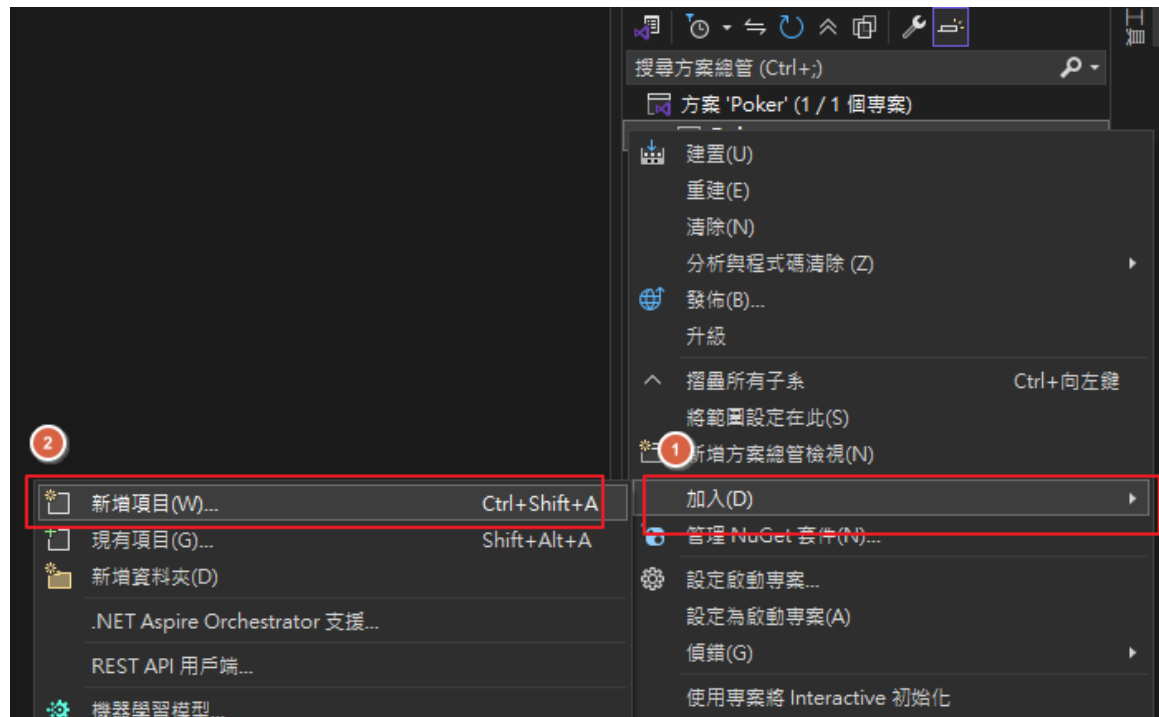
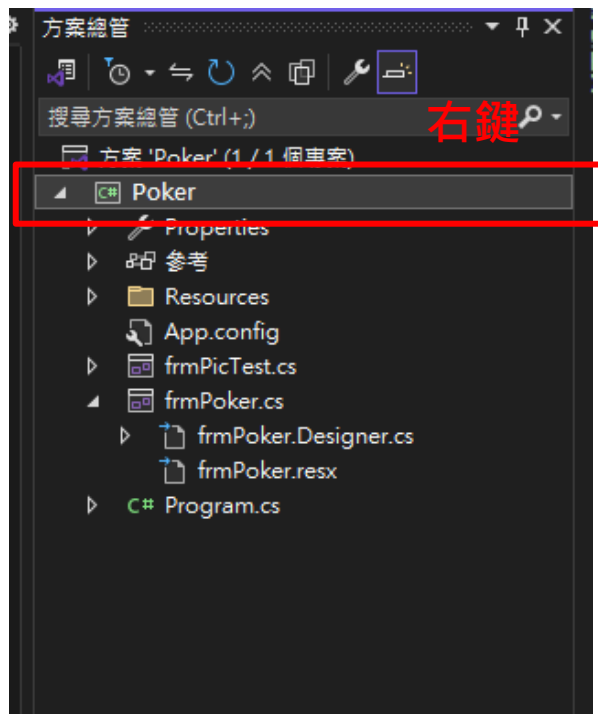
    picTest.Image = GetImage(picNum);

    lblNum.Text = picNum.ToString();
}
```

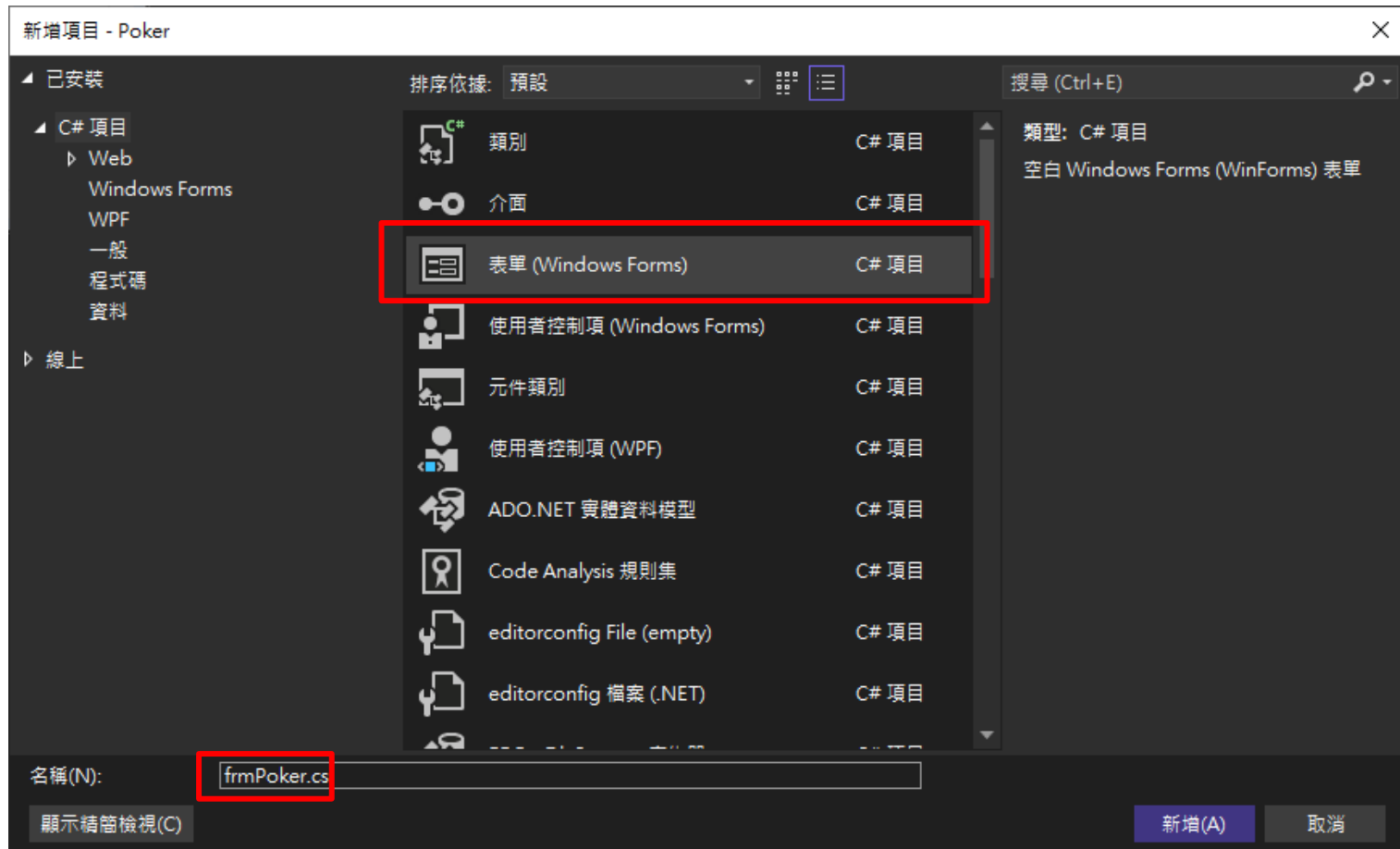
Section 4.2

動態控制項 (五張撲克牌)

增加新項目

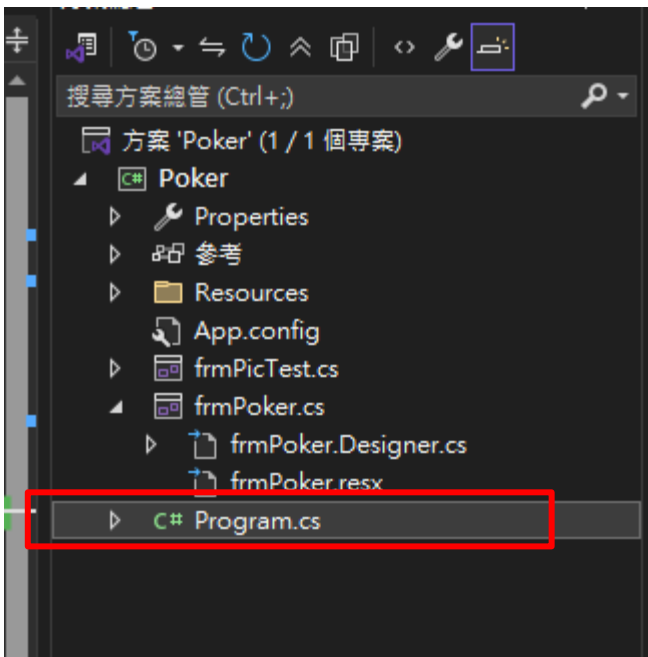


新增 Poker 表單



切換啟始表單

■ 切換啟始表單

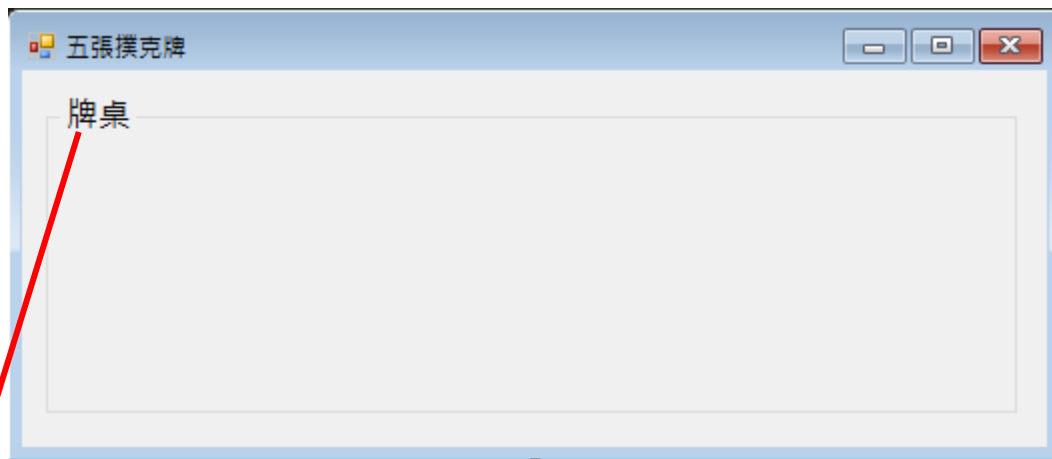


```
static void Main()
{
    Application.EnableVisualStyles();

    Application.SetCompatibleTextRendering
    Default(false);
    Application.Run(new frmPicTest());
}
```

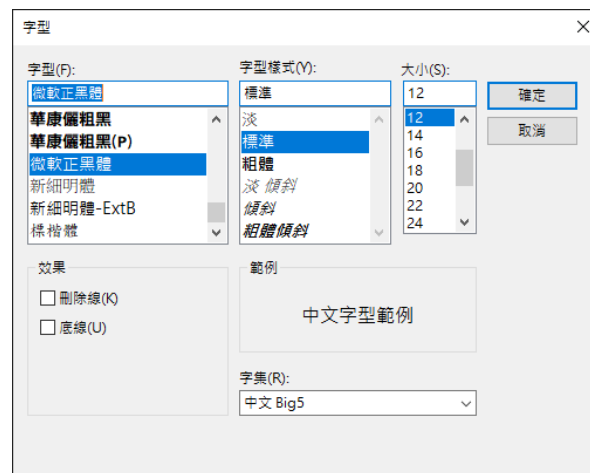
換成 `frmPoker`

設計牌桌



grpPoker

+	Padding	3, 3, 3, 3
	RightToLeft	No
+	Size	485, 160
	TabIndex	4
	Tag	
	Text	牌桌
	UseCompatibleTextF	False
	UseWaitCursor	False



設計五張牌牌桌

```
public partial class frmPoker : Form
{
```

```
    PictureBox[] pic = new PictureBox[5];
```

→ 控制項陣列

```
    public frmPoker()
```

```
{
```

```
        InitializeComponent();
```

```
        InitializePoker();
```

```
}
```

→ 動態產生撲克牌控制項函式

動態產生控制項

```
private void InitializePoker()  
{  
    // 動態產生5張牌  
    for (int i = 0; i < 5; i++) {  
        pic[i] = new PictureBox();  
        pic[i].Image = GetImage("back");  
        pic[i].Name = "pic" + i;  
        pic[i].SizeMode = PictureBoxSizeMode.AutoSize;  
        pic[i].Top = 30;  
        pic[i].Left = 10 + ((pic[i].Width + 10) * i);  
        pic[i].Visible = true;  
  
        // 將 pic 丟至到 grpPorker 內  
        this.grpPorker.Controls.Add(pic[i]);  
  
        pic[i].MouseClicked += new MouseEventHandler(pic_Click);  
    }  
}
```

預設顯示牌背

設定成自動大小

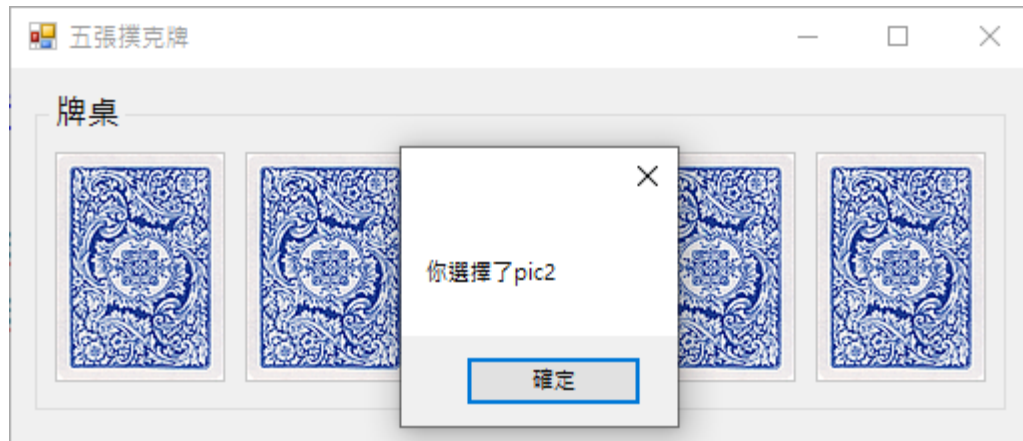
加入滑鼠事件

執行畫面

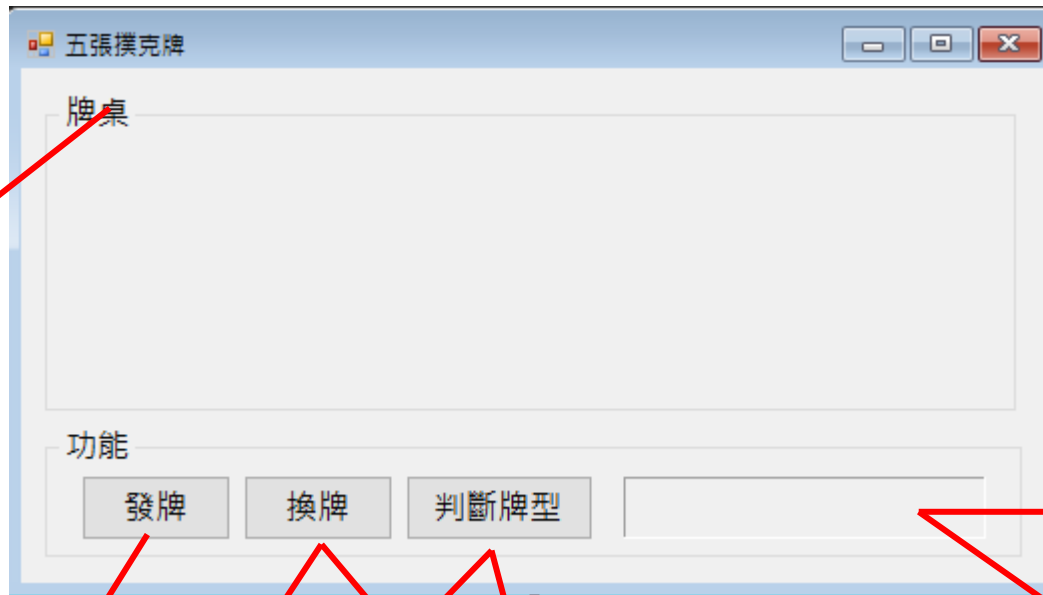


加入滑鼠點擊程式碼

```
private void pic_Click(object sender, MouseEventArgs e)
{
    PictureBox pic = (PictureBox)sender;
    MessageBox.Show("你選擇了" + pic.Name);
}
```



設計五張撲克牌介面



grpButton

lblResult

btnDealCard

btnChangeCard

btnCheck

ContextMenuStrip	(無)
Cursor	Default
DialogResult	None
Dock	None
Enabled	False
FlatAppearance	
FlatStyle	Standard
Font	微軟正黑體, 12pt

Size	181, 31
TabIndex	3
Tag	
Text	
TextAlign	MiddleCenter
UseCompatibleText	False
UseMnemonic	True
UseWaitCursor	False

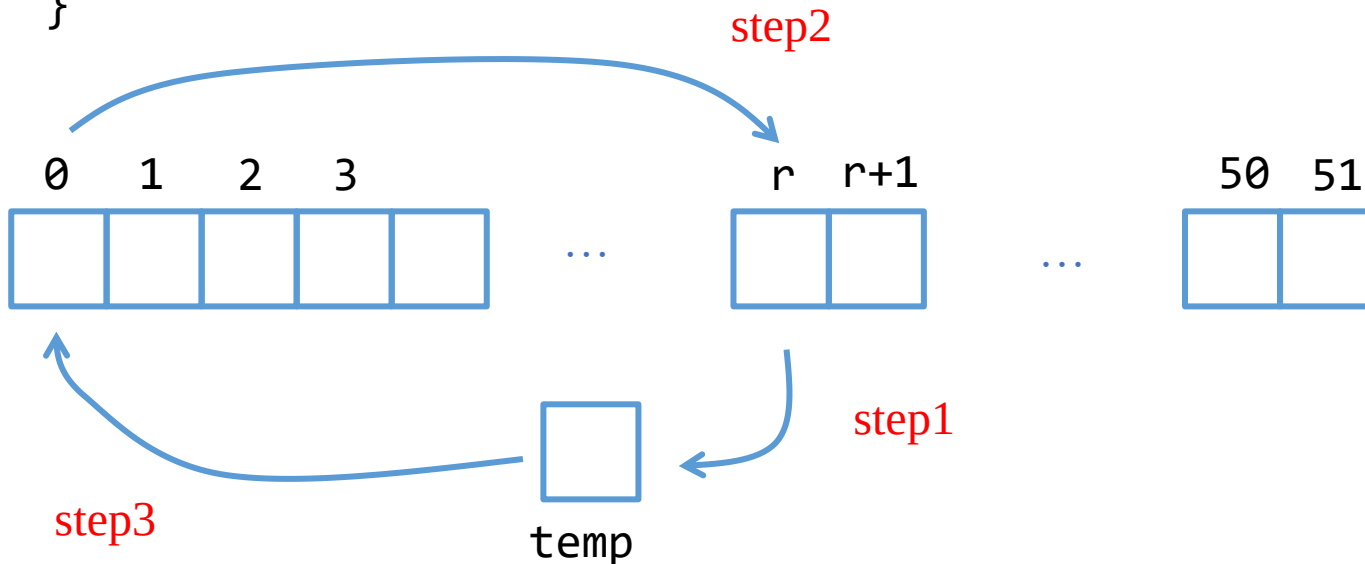
發牌

```
int[] allPoker = new int[52];
int[] playerPoker = new int[5];
private async void btnDealCard_Click(object sender, EventArgs e)
{
    // 先將牌面蓋掉
    for (int i = 0; i < 5; i++) {
        pic[i].Image = GetImage("back");
    }
    // 初始化52張牌
    for (int i = 0; i < 52; i++) {
        allPoker[i] = i;
    }
    // 洗牌
    Shuffle();
    // 發牌
    for (int i = 0; i < 5; i++) {
        pic[i].Image = GetImage("pic" + (allPoker[i] + 1));
        playerPoker[i] = allPoker[i];
    }
}
```

記錄目的牌號

洗牌

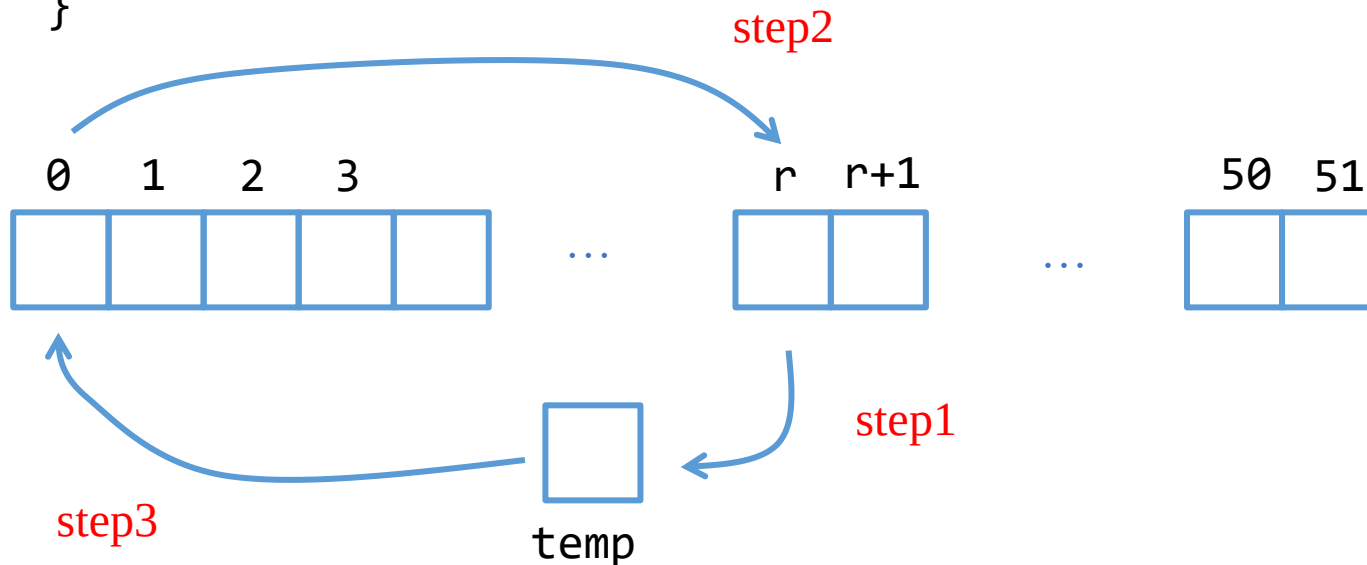
```
private void Shuffle()  
{  
    Random rand = new Random();  
    for (int i = 0; i < allPoker.Length; i++)  
    {  
        int r = rand.Next(allPoker.Length);  
step1 int temp = allPoker[r];  
step2 allPoker[r] = allPoker[0];  
step3 allPoker[0] = temp;  
    }  
}
```



洗牌

```
private void Shuffle()  
{  
    Random rand = new Random();  
    for (int i = 0; i < allPoker.Length; i++)  
    {  
        int r = rand.Next(allPoker.Length);  
        int temp = allPoker[r];  
        allPoker[r] = allPoker[0];  
        allPoker[0] = temp;  
    }  
}
```

可以執行500次嗎?



發牌執行畫面



執行速度過快，在蓋牌和發牌之間加入暫停

```
// 暫停500ms  
await Task.Delay(500);
```

設定選牌初始值

```
private void InitializePoker()  
{
```

```
    // 動態產生5張牌
```

```
    for (int i = 0; i < 5; i++)  
    {
```

```
        pic[i] = new PictureBox();  
        pic[i].Image = GetImage("back");  
        pic[i].Name = "pic" + i;  
        pic[i].SizeMode = PictureBoxSizeMode.AutoSize;  
        pic[i].Top = 30;  
        pic[i].Left = 10 + ((pic[i].Width + 10) * i);  
        pic[i].Visible = true;
```

```
        pic[i].Enabled = false;
```

```
        pic[i].Tag = "back";
```

```
        // 將 pic 丟至到 grpPoker 內
```

```
        this.grpPoker.Controls.Add(pic[i]);
```

```
        pic[i].MouseClicked += new MouseEventHandler(pic_Click);
```

```
    }
```

```
}
```

發牌之前，關閉 `pic[i]`，
以免誤按

記錄初始狀態為牌背

發牌完，啟用換按鈕

// 啟用所有牌的點擊事件

```
for (int i = 0; i < 5; i++)  
{  
    pic[i].Enabled = true;  
    pic[i].Tag = "front";  
}
```

發完牌後，啟用點擊事件，
好讓滑鼠選擇要換的牌

設定目前的牌已經翻開

```
btnChangeCard.Enabled = true;
```

將換牌按鈕啟用

當滑鼠點擊撲克牌選擇要換的牌

```
private void pic_Click(object sender, MouseEventArgs e)
{
    PictureBox pic = (PictureBox)sender;

    // 取得 pic 的索引值
    int index = int.Parse(pic.Name.Replace("pic", ""));

    // 如果 pic 的 Tag 為 back，則將顯示撲克牌
    if (pic.Tag.ToString() == "back") {
        pic.Tag = "front";
        pic.Image = GetImage(playerPoker[i] + 1);
    }
    else {
        pic.Tag = "back";
        pic.Image = GetImage("back");
    }
}
```




發牌



選擇要換的牌



取消第五張要換的牌

換牌

```
private void btnChangeCard_Click(object sender, EventArgs e)
```

```
{
```

```
    int cardIndex = 5;
```

```
    for (int i = 0; i < pic.Length; i++){  
        if (pic[i].Tag.ToString() == "back")
```

```
        {
```

```
            playerPoker[i] = allPoker[cardIndex];
```

```
            pic[i].Image = GetImage(playerPoker[i] + 1);
```

```
            pic[i].Tag = "front";
```

```
            cardIndex++;
```

```
        }
```

```
    }
```

```
    // 禁用所有牌的點擊事件
```

```
    for (int i = 0; i < pic.Length; i++) {  
        pic[i].Enabled = false;
```

```
    }
```

```
    btnChangeCard.Enabled = false;
```

```
    btnCheck.Enabled = true;
```

```
}
```

前五張已經發牌了，
為了避免重複，從第
六張換牌

換牌只能一次，
所以換完牌關
閉滑鼠點擊

將換牌按鈕停用

將判斷牌型按鈕啟用

計算花色和點數

```
string[] colorList = { "梅花", "方塊", "愛心", "黑桃" };  
string[] pointList = { "A", "2", "3", "4", "5", "6", "7", "8", "9", "10", "J", "Q",  
"K" };
```

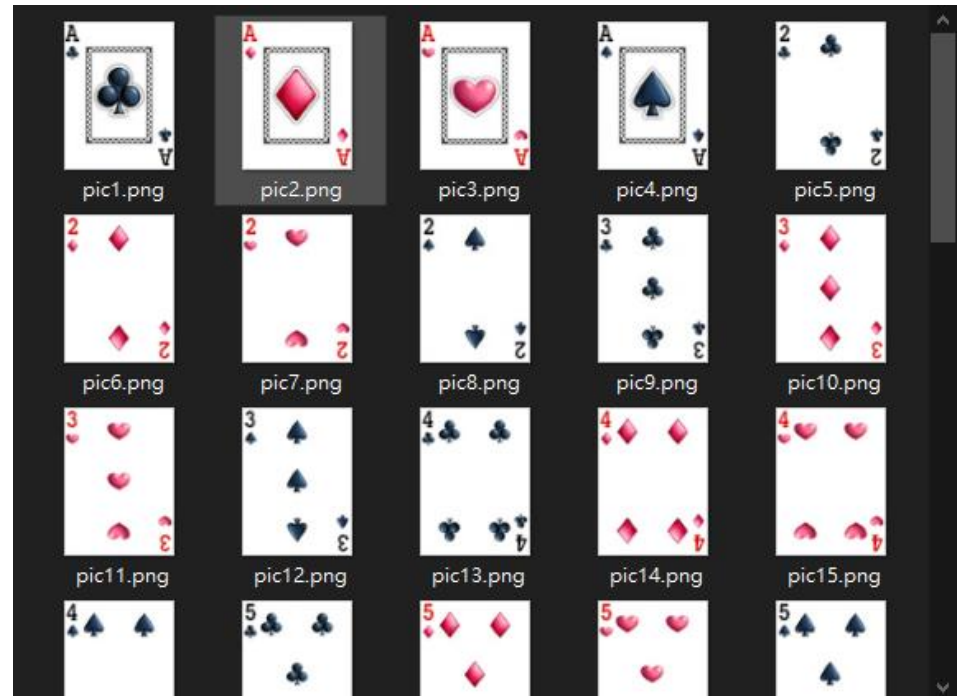
// 計錄目前五張撲克牌的花色和點數的陣列

```
int[] pokerColor = new int[5];
```

```
int[] pokerPoint = new int[5];
```

// 將每張牌的顏色和點數分別存入 **pokerColor** 和 **pokerPoint** 陣列

```
for (int i = 0; i < 5; i++) {  
    pokerColor[i] = playerPoker[i] % 4;  
    pokerPoint[i] = playerPoker[i] / 4;  
}
```



統計花色和點數並排序

```
// 記錄花色和點數出現次數的陣列
int[] colorCount = new int[4];
int[] pointCount = new int[13];

// 統計 color 和 point 出現次數
for (int i = 0; i < 5; i++) {
    int color = pokerColor[i];
    int point = pokerPoint[i];

    colorCount[color]++;
    pointCount[point]++;
}

// 排序 colorCount 和 pointCount 由大到小
Array.Sort(colorCount, colorList);
Array.Reverse(colorCount);
Array.Reverse(colorList);

Array.Sort(pointCount, pointList);
Array.Reverse(pointCount);
Array.Reverse(pointList);
```

Array.sort() 演算法

- 先以 快速排序 (Quicksort) 為主，若遞迴深度過深（分割區數目超過 $2 \times \log N$ ），會自動切換至 堆積排序 (Heapsort) 以避免最壞情況。
- 若元素個數小於或等於 16 個時，會改用 插入排序 (Insertion Sort)，因為小數據量下它的效率更高。

判斷牌型 (1)

```
// 判斷是否為同花
bool isFlush = (colorCount[0] == 5);
// 判斷是否為五張單張
bool isSingle = (pointCount[0] == 1 && pointCount[1] == 1 && pointCount[2] == 1 &&
pointCount[3] == 1 && pointCount[4] == 1);
// 判斷是否為差四
bool isDiffFout = (pokerPoint.Max() - pokerPoint.Min() == 4);
// 判斷是否為大順
bool isRoyal = pokerPoint.Contains(0) && pokerPoint.Contains(9) &&
pokerPoint.Contains(10) && pokerPoint.Contains(11) && pokerPoint.Contains(12);
// 判斷是否為同花大順
bool isRoyalisFlush = isFlush && isRoyal;
// 判斷是否為同花順
bool isStraightFlush = isFlush && isSingle && isDiffFout;
// 判斷是否為順子
bool isStraight = isSingle && (isDiffFout || isRoyal);
// 判斷是否為鐵支
bool isFourOfAKind = (pointCount[0] == 4);
// 判斷是否為葫蘆
bool isFullHouse = (pointCount[0] == 3 && pointCount[1] == 2);
// 判斷是否為三條
bool isThreeOfAKind = (pointCount[0] == 3 && pointCount[1] == 1);
// 判斷是否為兩對
bool isTwoPair = (pointCount[0] == 2 && pointCount[1] == 2);
// 判斷是否為一對
bool isOnePair = (pointCount[0] == 2 && pointCount[1] == 1);
```

判斷牌型 (2)

```
string result = "";
if (isRoyalisFlush) {
    result = $"{colorList[0]} 同花大順";
} else if (isStraightFlush) {
    result = $"{colorList[0]} 同花順";
} else if (isStraight) {
    result = "順子";
} else if (isFourOfAKind) {
    result = $"{pointList[0]} 鐵支";
} else if (isFullHouse) {
    result = $"{pointList[0]}三張{pointList[1]}兩張 葫蘆";
} else if (isFlush) {
    result = $"{colorList[0]} 同花";
} else if (isThreeOfAKind) {
    result = $"{pointList[0]} 三條";
} else if (isTwoPair) {
    result = $"{pointList[0]},{pointList[1]} 兩對";
} else if (isOnePair) {
    result = $"{pointList[0]} 一對";
} else {
    result = "雜牌";
}
lblResult.Text = result;
btnChangeCard.Enabled = false;
btnCheck.Enabled = false;
btnDealCard.Enabled = true;
```

判斷牌型 (3)



測試牌型

// 同花大順

```
playerPoker[0] = 51; playerPoker[1] = 47; playerPoker[2] = 43; playerPoker[3] = 39; playerPoker[4] = 3;
```

// 同花順

```
playerPoker[0] = 37; playerPoker[1] = 33; playerPoker[2] = 29; playerPoker[3] = 25; playerPoker[4] = 21;
```

// 同花

```
playerPoker[0] = 50; playerPoker[1] = 38; playerPoker[2] = 34; playerPoker[3] = 22; playerPoker[4] = 18;
```

// 鐵支

```
playerPoker[0] = 48; playerPoker[1] = 39; playerPoker[2] = 38; playerPoker[3] = 37; playerPoker[4] = 36;
```

// 葫蘆

```
playerPoker[0] = 30; playerPoker[1] = 29; playerPoker[2] = 6; playerPoker[3] = 5; playerPoker[4] = 4;
```

// 三條

```
playerPoker[0] = 48; playerPoker[1] = 39; playerPoker[2] = 15; playerPoker[3] = 14; playerPoker[4] = 13;
```

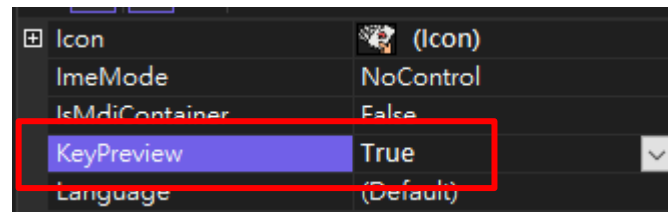
加入顯示牌型

```
/// <summary>
/// 顯示五張撲克牌到桌面上
/// </summary>
private void ShowCards()
{
    for (int i = 0; i < 5; i++)
    {
        pic[i].Image = this.GetImage($"pic{playerPoker[i] + 1}");
    }
}
```

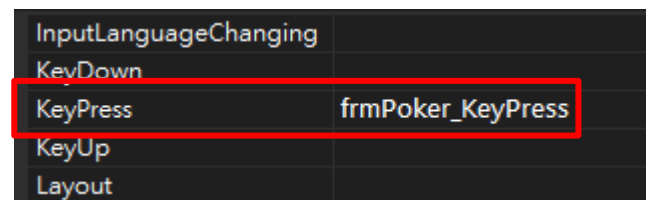
KeyPress 事件

■ 加入隱藏秘技按鍵

- 表單屬性的 KeyPreview 設成 True



- 表單事件的 KeyPress 加入程式碼



```

private void frmPoker_KeyPress(object sender,
KeyPressEventArgs e)
{
    if(btnDealCard.Enabled == false)
    {
        switch(e.KeyChar)
        {
            case 'q': // q鍵
                // 同花大順
                playerPoker[0] = 51;
                playerPoker[1] = 47;
                playerPoker[2] = 43;
                playerPoker[3] = 39;
                playerPoker[4] = 3;
                break;
            case 'w': // w鍵
                // 同花順
                playerPoker[0] = 37;
                playerPoker[1] = 33;
                playerPoker[2] = 29;
                playerPoker[3] = 25;
                playerPoker[4] = 21;
                break;
            case 'e': // e鍵
                // 同花
                playerPoker[0] = 50;
                playerPoker[1] = 38;
                playerPoker[2] = 34;
                playerPoker[3] = 22;
                playerPoker[4] = 18;
                break;
        }
    }
}

```

```

case 'r': // r鍵
    // 鐵支
    playerPoker[0] = 48;
    playerPoker[1] = 39;
    playerPoker[2] = 38;
    playerPoker[3] = 37;
    playerPoker[4] = 36;
    break;
case 't': // t鍵
    // 葫蘆
    playerPoker[0] = 30;
    playerPoker[1] = 29;
    playerPoker[2] = 6;
    playerPoker[3] = 5;
    playerPoker[4] = 4;
    break;
case 'y': // y鍵
    // 三條
    playerPoker[0] = 48;
    playerPoker[1] = 39;
    playerPoker[2] = 15;
    playerPoker[3] = 14;
    playerPoker[4] = 13;
    break;
}
// 顯示五張撲克牌到桌面上
ShowCards();

```